

Capítulo 06

Tokenization

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 30/07/2026

Capítulo 06 — Tokenization (BPE do zero)

Objetivo de aprendizagem: construir um tokenizador **BPE** (*byte pair encoding*) do zero — o mesmo algoritmo que o GPT usa para transformar texto em tokens. No caminho, entender **Unicode** e **UTF-8**, por que tokenizadores sérios começam pelos **bytes**, e descobrir (medindo) por que escrever em português custa mais caro numa API de LLM.

Pré-requisitos: Capítulos 1-5. Este capítulo não usa PyTorch nem treina redes — é Python puro, e roda em segundos. Mas é uma das peças mais consequentes de um LLM.

Arquivos: - `unicode_utf8.py` — os fundamentos: caractere -> code point -> bytes - `bpe.py` — o tokenizador BPE completo (treino, `encode`, `decode`) - `exercicios.md` — exercícios

1. O problema

Até aqui usamos **um token por caractere**: 27 tokens (26 letras + fronteira). É simples e funciona, mas tem um custo sério. Considere as duas alternativas óbvias:

Abordagem	Vocabulário	Problema
Um token por caractere	~27 (ou ~150 mil, com Unicode)	sequências longuíssimas
Um token por palavra	centenas de milhares	palavras novas viram "desconhecido"

Por que sequências longas são ruins? Lembre do Capítulo 4: a atenção compara todas as posições com todas as outras, então o custo cresce com **T²** no tamanho do contexto. Se cada letra é um token, "informação" gasta 10 posições. Dobrar o número de tokens por texto **quadruplica** o custo da atenção — e reduz quanto texto real cabe na janela de contexto.

Por que vocabulário de palavras é ruim? Além do tamanho, existe o problema de palavras nunca vistas (*out-of-vocabulary*). O que o modelo faz com "criptomoeda" se ela não estava no vocabulário? E com um nome próprio novo? E com uma palavra em japonês?

O BPE resolve os dois de uma vez, com uma ideia simples: **comece pelos bytes e vá juntando os pares mais frequentes**.

2. Antes do BPE: como texto vira número

Precisamos de três níveis bem claros. Rode `python unicode_utf8.py`.

Nível 1 — code point

O **Unicode** atribui um número (*code point*) a cada caractere existente:

```
'a' -> code point 97 (U+0061)
'é' -> code point 233 (U+00E9)
chr(26085) -> code point 26085 (U+65E5) # ideograma japonês
chr(128640) -> code point 128640 (U+1F680) # emoji de foguete
```

Nota sobre esta apostila: os dois últimos exemplos aparecem como `chr(...)` porque as fontes deste PDF não cobrem ideogramas nem emoji. No terminal, ao rodar `unicode_utf8.py`, você vê os caracteres de verdade. A limitação é da fonte do documento, não do código — e é, ela mesma, um exemplo do assunto do capítulo.

São ~150 mil caracteres definidos. Um token por caractere daria um vocabulário gigantesco — e ainda assim incompleto, porque o Unicode cresce.

Nível 2 — bytes (UTF-8)

Code points são conceitos; para gravar em memória usamos uma **codificação**. O UTF-8 usa um número **variável** de bytes:

```
'a' -> 1 byte(s): [97]
'é' -> 2 byte(s): [195 169]
chr(26085) -> 3 byte(s): [230 151 165] # ideograma japonês
chr(128640) -> 4 byte(s): [240 159 154 128] # emoji
```

O imposto do português

Aqui aparece um fato que nos afeta diretamente: **caracteres acentuados custam 2 bytes**.

```
acao 4 bytes | ação 6 bytes (+2)
informacao 10 bytes | informação 12 bytes (+2)
coracao 7 bytes | coração 9 bytes (+2)
```

A frase `"A ação começa às três horas."` tem 28 caracteres mas **33 bytes**. Guarde isso — vamos medir a consequência prática na Seção 7.

Nível 3 — por que começar pelos bytes

Existem apenas **256** valores possíveis de byte. Isso dá um vocabulário base que é ao mesmo tempo **pequeno** e **completo**: qualquer texto, de qualquer idioma, mais emoji, mais código-fonte, é representável como bytes. **Nunca existe um token "desconhecido"**.

Essa garantia é o alicerce do BPE.

3. O algoritmo BPE

A ideia inteira em três linhas:

1. Comece com um token por byte (256 tokens).

2. Encontre o **par de tokens vizinhos mais frequente** no corpus.
3. Funda esse par num **token novo**. Repita.

Um exemplo à mão, com o texto `"aa ab aa ab"`:

```
inicial: a a _ a b _ a a _ a b o par ('a','a') aparece 2x -> vira o token X
depois: X _ a b _ X _ a b o par ('a','b') aparece 2x -> vira o token Y
depois: X _ Y _ X _ Y nada mais se repete
```

Cada fusão **encurta** a sequência e **acrescenta** um token ao vocabulário. Você controla o quanto quer disso pelo `vocab_size`.

As duas funções que fazem tudo

```
def get_stats(ids):
    """Conta quantas vezes cada par de tokens vizinhos aparece."""
    counts = {}
    for pair in zip(ids, ids[1:]):
        counts[pair] = counts.get(pair, 0) + 1
    return counts

def merge(ids, pair, new_id):
    """Substitui toda ocorrência de `pair` pelo token `new_id`."""
    out = []
    i = 0
    while i < len(ids):
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i + 1] == pair[1]:
            out.append(new_id)
            i += 2 # salta os DOIS tokens fundidos
        else:
            out.append(ids[i])
            i += 1
    return out
```

O `i += 2` importa: depois de fundir um par, pulamos os dois elementos consumidos. Sem isso, em `"aaa"` a mesma letra do meio participaria de duas fusões.

O laço de treino

```
ids = list(text.encode("utf-8")) # começa nos bytes crus
self.vocab = {i: bytes([i]) for i in range(256)}

for k in range(vocab_size - 256):
    stats = get_stats(ids)
    pair = max(stats, key=stats.get) # o par mais frequente
    new_id = 256 + k
    ids = merge(ids, pair, new_id)
    self.merges[pair] = new_id
    self.vocab[new_id] = self.vocab[pair[0]] + self.vocab[pair[1]]
```

Note a última linha: o significado de um token novo é a **concatenação dos bytes dos seus dois pais**. É assim que o vocabulário se constrói recursivamente — um token pode ser feito de tokens que

também foram fundidos antes.

4. `encode` e a importância da ordem

Para tokenizar um texto novo, aplicamos as fusões aprendidas. Mas **na ordem em que foram aprendidas**:

```
def encode(self, text):
    ids = list(text.encode("utf-8"))
    while len(ids) >= 2:
        stats = get_stats(ids)
        # entre os pares presentes, escolhe o que foi aprendido MAIS CEDO
        pair = min(stats, key=lambda p: self.merges.get(p, float("inf")))
        if pair not in self.merges:
            break
        ids = merge(ids, pair, self.merges[pair])
    return ids
```

Por que a ordem é obrigatória? Porque as fusões são **dependentes**: a fusão nº 7 do nosso treino foi `('on', '\n') -> 'on\n'`, e o token `'on'` só existe porque a fusão nº 5 o criou. Aplicar a nº 7 antes da nº 5 seria impossível — o token nem existiria. O `min(...)` com o índice da fusão garante essa disciplina. (No exercício E4 você quebra isso de propósito e observa o resultado.)

O `decode` é o caminho inverso, e tem um detalhe:

```
def decode(self, ids):
    bs = b"".join(self.vocab[i] for i in ids)
    return bs.decode("utf-8", errors="replace")
```

O `errors="replace"` é necessário porque **um token pode conter meio caractere**. Se `'é'` são os bytes `[195, 169]` e o tokenizador nunca aprendeu a juntá-los, cada byte é um token — e isolado, nenhum deles é UTF-8 válido. Sem o `errors="replace"`, o programa quebraria.

5. Rodando: o que o tokenizador aprendeu

Rodando `python bpe.py` (leva ~8 segundos, com `vocab_size = 512` sobre 150 mil caracteres de nomes):

```
fusao 1: (97, 110) -> 256 ('an'), 4608x
fusao 2: (97, 10) -> 257 ('a\n'), 4239x
fusao 3: (101, 10) -> 258 ('e\n'), 3878x
fusao 4: (101, 108) -> 259 ('el'), 3405x
fusao 5: (111, 110) -> 260 ('on'), 2610x

bytes originais : 150000
tokens depois : 66936
taxa de compressao: 2.24x
```

As fusões contam uma história. 'an' é a primeira porque é a sequência mais comum em nomes brasileiros. Várias fusões iniciais terminam em \n ('a\n', 'e\n', 'on\n') porque nosso arquivo tem **um nome por linha** — o modelo está aprendendo *terminações de nome*, e o fim de linha faz parte do padrão.

E os tokens mais longos que ele criou são reveladores:

```
token 334 = 'ilson\n'
token 348 = 'erson\n'
token 374 = 'isson\n'
token 412 = 'ilton\n'
token 463 = 'ilane\n'
token 362 = 'iana\n'
```

Ninguém ensinou morfologia ao algoritmo. Ele descobriu sozinho, apenas contando pares, que -ilson, -erson, -ilton e -iana são sufixos produtivos de nomes brasileiros. É estatística pura virando estrutura linguística.

A propriedade que não pode falhar

Um tokenizador precisa ser **reversível**: `decode(encode(x)) == x`, sempre, para qualquer entrada. Testamos casos difíceis de propósito:

```
[OK ] 'maria eduarda' 13 bytes -> 7 tokens (1.86x)
[OK ] 'josé da conceição' 20 bytes -> 16 tokens (1.25x)
[OK ] 'A ação começa às três.' 27 bytes -> 27 tokens (1.00x)
[OK ] 'Olá! ' + japones + emoji 17 bytes -> 17 tokens (1.00x)
[OK ] '' 0 bytes -> 0 tokens
[OK ] 'xyzkw' 5 bytes -> 5 tokens (1.00x)

TODOS os round-trips passaram? True
```

Repare em duas coisas. Primeiro: **japones e emoji funcionam**, embora não existisse nada parecido no treino — é a garantia dos bytes em ação. Segundo: a compressão nesses casos é **1.00x**, ou seja, **nenhuma**. O tokenizador não comprime o que nunca viu; ele só não quebra.

6. Compressão só existe dentro do domínio

Texto	Compressão
Nomes usados no treino	2,24x
Nomes novos (não vistos)	2,17x
Frase em português corrido	1,20x
Japonês + emoji	1,00x

A compressão se sustenta em nomes novos (2,17x contra 2,24x) — ele aprendeu o *padrão*, não decorou a lista. Mas cai para 1,20x numa frase comum, porque ele foi treinado em **nomes**, não em prosa.

Regra prática: tokenizador e dados andam juntos. Um tokenizador é um modelo estatístico do seu corpus — treiná-lo no domínio errado desperdiça vocabulário.

7. O imposto do português, medido

Aqui a teoria vira dinheiro. Veja como a frase "A informação sobre a ação de coração está na página." é fatiada pelo nosso tokenizador de nomes:

A | | in | f | or | ma | <0xC3> | <0xA7> | <0xC3> | <0xA3> | o | | s | o | b | re | ...

São **50 tokens** para 60 bytes, e **16 deles são fragmentos de byte** (<0xNN>) — todos vindos de caracteres acentuados. O **ção** sozinho consome 5 tokens (<0xC3> <0xA7> <0xC3> <0xA3> o) para representar 3 caracteres.

A solução do exercício E5 treina um segundo tokenizador, do **mesmo tamanho**, em texto português de verdade (as próprias apostilas deste curso). Ele aprende:

```
token 371 = 'ação '  
token 317 = 'ção '  
token 471 = 'são '  
token 485 = 'não '  
token 287 = 'ão '
```

E a mesma frase passa a ser fatiada assim:

A | in | form | ação | so | b | r | e | a | ação | de | c | or | ação | est | á | ...

25 tokens em vez de 50. Medindo em três frases:

Tokenizador	Total de tokens
Treinado em nomes (sem acento)	135
Treinado em português	64

53% menos tokens para o mesmo texto, com vocabulário do mesmo tamanho.

Isso tem consequência direta: APIs de LLM cobram **por token**, e a janela de contexto é medida **em tokens**. Um tokenizador treinado majoritariamente em inglês — como os dos modelos comerciais — faz o português custar mais caro e ocupar mais contexto pelo mesmo conteúdo. Não é impressão: é aritmética de bytes.

8. Resumo do capítulo

- Tokenizar é escolher a **unidade** do modelo. Caracteres dão sequências longas (custo T^2 na atenção); palavras dão vocabulário enorme e o problema de palavras novas.
- **Unicode** dá um número a cada caractere; **UTF-8** grava esse número em 1-4 bytes. Acentos custam 2 bytes — o "imposto do português".
- Começar pelos **bytes** (256 tokens) garante que **qualquer** texto é representável: nunca há "desconhecido".
- **BPE**: funda repetidamente o par de tokens vizinhos mais frequente. `get_stats` + `merge` são o algoritmo inteiro.
- O `encode` deve aplicar as fusões **na ordem em que foram aprendidas**, porque elas são dependentes entre si.
- `decode` precisa de `errors="replace"`: um token pode conter meio caractere.
- **Reversibilidade** (`decode(encode(x)) == x`) é obrigatória — verificamos com acentos, japonês, emoji e string vazia.
- Compressão medida: **2,24x** no domínio, **2,17x** fora dele, **1,00x** em texto totalmente estranho. Tokenizador é um modelo do corpus.
- Treinar o tokenizador no idioma certo economizou **53% dos tokens** — com impacto direto em custo de API e contexto útil.

O que vem no Capítulo 7

Já temos arquitetura (Cap. 5) e entrada (Cap. 6). Falta treinar **bem**. No **Capítulo 07 — Optimization** vamos olhar o que até agora tratamos com valores "razoáveis" escolhidos na mão: **inicialização** dos pesos, o otimizador **AdamW** por dentro, agendamento da **learning rate** e *gradient clipping*. É a diferença entre um treino que funciona e um treino que funciona bem.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 06 (Tokenization)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

E1 — Unicode e UTF-8 (aquecimento)

Sem rodar código: 1. Quantos **bytes** ocupa "pão" em UTF-8? E "pao"? Por quê? 2. `ord("ç")` vale 231. Isso significa que "ç" cabe em 1 byte? Explique. 3. Por que um tokenizador que começa pelos **bytes** nunca encontra um caractere "desconhecido"?

E2 — O tamanho do vocabulário

Rode `bpe.py` com `VOCAB_SIZE = 300, 512 e 1024`. 1. Como muda a **taxa de compressão** em cada caso? 2. O ganho é linear? Onde começa o retorno decrescente? 3. Qual o custo de um vocabulário maior? (Pense na camada de saída do modelo: a `lm_head` do Capítulo 5 tem `n_embd × vocab_size` parâmetros.)

E3 — As primeiras fusões

Olhe a lista de fusões que o `bpe.py` imprime (as 10 primeiras). 1. Por que `('a', 'n') -> 'an'` é a primeira? O que isso diz sobre nomes brasileiros? 2. Várias fusões iniciais terminam em `\n` (fim de linha), como `'a\n'` e `'on\n'`. Por quê? O que isso revela sobre o formato do nosso arquivo de treino? 3. Entre os tokens mais longos aparecem `'ilson'`, `'erson'`, `'ilton'`. O tokenizador descobriu **sufixos de nomes** sem ninguém ensinar. Como?

E4 — A ordem das fusões importa

No método `encode`, trocamos a escolha do par por `min(stats, key=...)` usando a ordem de aprendizado. Experimente trocar por uma escolha arbitrária (por exemplo, o primeiro par encontrado que esteja em `self.merges`). 1. O `decode(encode(x)) == x` continua valendo? 2. A compressão fica pior? Por quê a ordem de aprendizado é a ordem correta? 3. Dica: uma fusão tardia pode usar um token criado por uma fusão anterior. O que acontece se você aplicar a tardia primeiro?

E5 — O imposto do português (importante)

O `bpe.py` mostra que "A informação sobre a ação..." é fatiada com 16 tokens de fragmentos de byte (`<0xNN>`), porque o tokenizador foi treinado em nomes **sem** acento. 1. Treine um BPE **em texto português com acentos** e verifique se ele aprende `'ção'` (ou `'ão'`) como um token só. Use como corpus os próprios `README.md` do curso. 2. Compare a contagem de tokens da mesma frase nos dois tokenizadores. 3. Isso tem consequência prática real: APIs de LLM cobram **por token**. Se o tokenizador foi treinado majoritariamente em inglês, escrever em português custa mais. Estime esse sobrecusto a partir dos seus números.

E6 — Tokens fora do domínio

Note que o caso com japonês e emoji (veja [bpe.py](#)) teve compressão **1.00x** — nenhuma. 1. Por que o tokenizador não comprime nada nesse caso? 2. Ele ainda consegue **representar** o texto? (Rode e confirme.) Por que isso é uma garantia importante? 3. O que aconteceria com um tokenizador baseado em **palavras** ao receber uma palavra em japonês?

E7 — Integrando com o modelo (desafio, conecta ao Cap. 5)

Nosso Transformer do Capítulo 5 usa 27 tokens (um por letra). Adapte-o para usar o tokenizador BPE deste capítulo. 1. O que muda em [vocab_size](#) e, por consequência, no número de parâmetros? 2. Com tokens maiores, cada posição carrega mais informação. Um [block_size](#) de 8 passa a cobrir mais texto — quanto mais, aproximadamente, dada a taxa de compressão? 3. A loss fica comparável à do Capítulo 5? **Cuidado:** ela **não** é diretamente comparável. Explique por que uma loss por *token* com vocabulário diferente não pode ser comparada diretamente. (Dica: prever entre 27 opções e prever entre 512 é uma tarefa de dificuldade diferente.)